

Modeling of an Algae Pond

Octave Litrico

January 2026

Introduction

In the context of modeling an algae cultivation pond, the evolution of a scalar quantity $\rho(x, t)$, representing the concentration of algae can be described by a partial differential equation of the advection-diffusion type:

$$\frac{\partial \rho}{\partial t} + a \frac{\partial \rho}{\partial x} = \mu \frac{\partial^2 \rho}{\partial x^2}$$

where:

- a is the advection (transport) coefficient related to the movement induced by the water wheel
- μ is the diffusion coefficient (natural or turbulent dispersion).

However, to simplify the numerical analysis and better understand diffusion phenomena alone, we choose in this work to neglect the advection term. This amounts to considering only a pure diffusion equation:

$$\frac{\partial \rho}{\partial t} = \mu \frac{\partial^2 \rho}{\partial x^2}$$

1 The discretized Model

1.1 Discretization principle

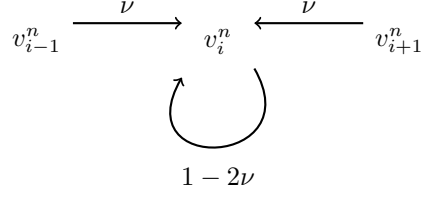
Let us discretize the time and space intervals. Let $N, M \in \mathbb{N}$, $h = \Delta_x = \frac{L}{N}$, $\Delta_t = \frac{T}{M}$ and $x_i = i\Delta_x$, $t_n = n\Delta_t \forall i \in \{0, \dots, N\}, n \in \{0, \dots, M\}$.

The pond will be modeled by a rectangle of size $L \times H$ divided into I horizontal layers of same size. The pond is circular, therefore the concentration functions of each layers will have to be periodic with period L . A water wheel located on the x -coordinate 0 stirs the water and will be modeled by a matrix P . It will intervene in the computing of the values of the algae concentration near 0. So that there is no production nor loss of matter, P must be column-stochastic. Indeed, we have that for all $i, j \in \{1, \dots, I\}$, $P_{i,j}$ is the amount of matter in line j sent to line i . Therefore, we must have that for all $j \in \{1, \dots, I\}$, $\sum_{k=1}^I P_{k,j} = 1$

The concentration function ρ will be an element of $\mathcal{C}^1([0, T], H^1([0, L])^I)$. At each time $t_n = n\Delta_t$, the concentration in cell j is denoted $\rho_j^n = \rho(t_n, x_j)$.

1.2 Cellular automat

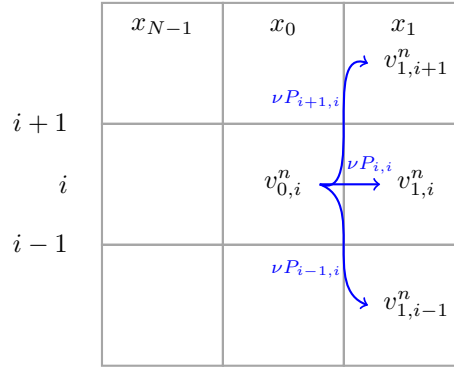
To first understand the underlying motion of the pond, let us introduce a cellular model. With the discretization showed above, we consider a cell in which the algae concentration evolves as a particle. A cell located far from the paddle wheel exchanges mass with its two neighbors. The scheme can be represented as follows:



Here, the value at time $n + 1$ is a weighted linear combination of the cell and its neighbors. ν here represents the mass proportion that each cell exchanges.

1.3 Special case: near the paddle wheel

Near the paddle wheel, the stir influences the concentrations. The following scheme represents the evolution, with the wheel located in column 0 :



Therefore we end up with this automat :

$$\begin{aligned}
& \forall n \in \{0, \dots, M-1\} \\
& \begin{cases}
v_j^{n+1} = \nu(v_{j-1}^n + v_{j+1}^n) + (1 - 2\nu)v_j^n & \forall j \in \{2, \dots, N-2\} \\
v_{N-1}^{n+1} = \nu(v_{N-2}^n + P v_0^n) + (1 - 2\nu)v_{N-1}^n \\
v_0^{n+1} = \nu(v_{N-1}^n + v_1^n) + (1 - 2\nu)P v_0^n \\
v_1^{n+1} = \nu(P v_0^n + v_2^n) + (1 - 2\nu)v_1^n
\end{cases}
\end{aligned}$$

2 PDE analysis

Let $\rho(t, x) \in \mathbb{R}^I$ be the vector of algae concentration on the x-coordinate of the pond at time t. We have to take into account the L -periodicity :

$$\rho(t, 0) = \rho(t, L) \quad \forall t$$

Furthermore, the total algae mass in the pond is constant through time. Therefore, we have :

$$\sum_i \int_0^L \rho_i = 0$$

up to a translation of the values of the ρ_i by the average concentration.

Let us denote U the set :

$$\{v \in H^2([0, L])^I \mid v(0) = v(L) = P v(L) \text{ and } \sum_{i=1}^I (\partial_x v_i(L) - \partial_x v_i(0)) = 0\}$$

and

$$V \text{ an eigen vector of } P \text{ such that } P V = V \text{ and } \|V\|_1 = 1$$

The ponctual values of $\partial_x u$ are well defined. Indeed, thanks to the Morrey injection, we have that $H^2([0, L]) \hookrightarrow \mathcal{C}^1([0, L])$. U is a Hilbert space for the following inner product :

$$\langle u, v \rangle_{U,U} = \sum_{i=1}^I \frac{1}{V_i} \int_0^L \partial_x u_i \cdot \partial_x v_i$$

The problem we are looking to solve is :

$$\begin{cases} \text{Find } \rho \in U \\ \text{such that } \partial_t \rho = \mu \Delta \rho \end{cases}$$

Firstly, let us notice that :

$$\begin{aligned} \forall v \in U, \\ \langle -\Delta \rho, v \rangle_{U,U} &= - \sum_{i=1}^I \frac{1}{V_i} \int_0^L \partial_{xx} \rho_i v \\ &= - \sum_{i=1}^I \frac{1}{V_i} \left[(\partial_x \rho_i(0) v_i(0) - \partial_x \rho_i(L) v_i(L)) - \int_0^L \partial_x \rho_i \partial_x v_i \right] \\ &= - \sum_{i=1}^I \left(\partial_x \rho_i(0) - \partial_x \rho_i(L) - \int_0^L \partial_x \rho_i \partial_x v_i \right) \quad \text{because } v_i(0) = v_i(L) = V_i \\ &= \sum_{i=1}^I \int_0^L \partial_x \rho_i \partial_x v_i \end{aligned}$$

We would like to find a Hilbert basis of U which will ease our difficulties. Let us introduce $H = \{h \in L^2([0, L])^I \mid \sum_i \int_0^L h_i = 0\}$, we can easily show that U is dense in H and by using the Rellich theorem, it follows that the embedding of U in H is compact.

Let us define for all $u, v \in U$:

$$a(u, v) = \mu \sum_{i=1}^I \int_0^L \partial_x u_i \cdot \partial_x v_i$$

The operator a is symmetric, bilinear, and coercive on U by the Poincaré inequality. Moreover, it is continuous. Therefore, there exists an infinite sequence $(\lambda_n)_{n \in \mathbb{N}} \in \mathbb{N}$ of positive eigenvalues of this operator, and a Hilbert basis $(x \mapsto \rho_n(x))_{n \in \mathbb{N}} \in U^{\mathbb{N}}$ of H (for the inner product $\langle \cdot, \cdot \rangle_{H,H}$) of associated eigenvectors. Let's name them $(\rho_n)_{n \in \mathbb{N}}$. The theorem also holds that the sequence $(\frac{\rho_n}{\sqrt{\lambda_n}})$ is a Hilbert basis of U for the inner product $a(\cdot, \cdot)$.

If ρ is solution to (1) then there exists a sequence $(t \mapsto \nu_n(t))_{n \in \mathbb{N}}$ of functions of $H^2([0, T])$ such that:

$$\rho(t, x) = \sum_{n \in \mathbb{N}} \nu_n(t) \rho_n(x)$$

The values of ρ_n and of ν_n must be found.

$$-\mu \Delta u = \lambda u$$

with $\lambda > 0$ So

$$u(x) = A \cos \left(\sqrt{\frac{\lambda}{\mu}} x \right) + B \sin \left(\sqrt{\frac{\lambda}{\mu}} x \right)$$

with $A, B \in \mathbb{R}^I$

Moreover, we know that $u(0) = Pu(0)$ and that P is irreducible and thus has an eigenspace associated to the eigenvalue 1 of dimension 1. Let us denote V a vector of norm 1 of this space. As each V_i is positive (given the ergodic theorem) we have $\|V\|_1 = \sum_i V_i = 1$. Then, $A = \alpha V$ with $\alpha \in \mathbb{R}$

With $u_i(0) = u_i(L)$ we see that :

$$B \sin \left(\sqrt{\frac{\lambda}{\mu}} L \right) = A \left[1 - \cos \left(\sqrt{\frac{\lambda}{\mu}} L \right) \right]$$

If $\sqrt{\frac{\lambda}{\mu}} L \notin \pi\mathbb{Z}$ then :

$$B = \alpha V \frac{(1 - \cos(\sqrt{\frac{\lambda}{\mu}} L))}{\sin(\sqrt{\frac{\lambda}{\mu}} L)}$$

Because $\sum_i \int_0^L \rho_i = 0$ we have :

$$\sum_i \left[A_i \sin \left(\sqrt{\frac{\lambda}{\mu}} L \right) + B_i \left(1 - \cos \left(\sqrt{\frac{\lambda}{\mu}} L \right) \right) \right] = 0$$

i.e

$$\alpha \left(1 - \cos \left(\sqrt{\frac{\lambda}{\mu}} L \right) \right)^2 = -\alpha \sin \left(\sqrt{\frac{\lambda}{\mu}} L \right)^2$$

therefore, as $\alpha \neq 0$ because $u \neq 0$, we have :

$$(1 - \cos(\sqrt{\frac{\lambda}{\mu}} L))^2 = -\sin^2(\sqrt{\frac{\lambda}{\mu}} L)$$

We end up with the following equation :

$$\cos \left(\sqrt{\frac{\lambda}{\mu}} L \right) = 1$$

i.e $\sqrt{\frac{\lambda}{\mu}} L \in 2\pi\mathbb{Z}$, contradiction.

Therefore, $\sqrt{\frac{\lambda}{\mu}} L \in \mathbb{N}^* \pi$

$\sqrt{\frac{\lambda}{\mu}} L = n\pi$ with $n \in \mathbb{Z}^*$.

- If $n \in 2\mathbb{N}^*$ then the equation $\sum_i \int_0^L \rho_i = 0$ is trivial, and there is no constraint on the B_i .
- If $n \in 2\mathbb{N}^* + 1$ then we have :

$$\sum_i B_i = 0$$

and :

$$A(1 - \cos(\sqrt{\frac{\lambda}{\mu}} L)) = 2A = 0$$

so

$$\alpha = 0$$

We have eventually found all the eigen vectors of the operator a . These are the elements of the following sets :

$$\begin{aligned}
& - \{x \mapsto V \cos(\frac{2n\pi x}{L})\}_{n \in \mathbb{N}^*} \\
& - \{x \mapsto e_1 \sin(\frac{2n\pi x}{L})\}_{n \in \mathbb{N}^*} \\
& - \dots \\
& - \{x \mapsto e_I \sin(\frac{2n\pi x}{L})\}_{n \in \mathbb{N}^*} \\
& - \left\{x \mapsto \sum_{i=1}^I \gamma_{2n+1}^i e_i \sin(\frac{(2n+1)\pi x}{L}) \mid \sum_{i=1}^I \gamma_{2n+1}^i = 0\right\}_{n \in \mathbb{N}^*}
\end{aligned}$$

As shown before, these vectors form a Hilbert basis of U equipped with the inner product $a(.,.)$ up to normalization. Therefore we can decompose ρ in this basis : there exist $\mathcal{C}^1([0, T], \mathbb{R})$ functions $\alpha_{2n}, \beta_{2n}^i, \gamma_{2n+1}^i \forall n \in \mathbb{N}^*, i \in \{1, \dots, I\}$ such that

$$\rho(x, t) = \sum_{n \in \mathbb{N}^*} \left[\alpha_{2n}(t) V \cos\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^I \beta_{2n}^i(t) e_i \sin\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^I \gamma_{2n+1}^i(t) e_i \sin\left(\frac{(2n+1)\pi x}{L}\right) \right]$$

with $\sum_{i=1}^I \gamma_{2n+1}^i(t) = 0 \forall t$

Therefore for all $n \in \mathbb{N}^*, i \in \{1, \dots, I\}$, $\alpha_{2n}, \beta_{2n}^i, \gamma_{2n+1}^i$ satisfy the respective equations

$$\begin{cases} \alpha'_{2n}(t) = \frac{-4n^2\pi^2\mu}{L^2} \alpha_{2n}(t) \\ \beta_{2n}^{i'}(t) = \frac{-4n^2\pi^2\mu}{L^2} \beta_{2n}^i(t) \\ \gamma_{2n+1}^{i'}(t) = \frac{-(2n+1)^2\pi^2\mu}{L^2} \gamma_{2n+1}^i(t) \end{cases} \quad \text{for all } t \in [0, T]$$

Therefore,

$$\begin{cases} \alpha_{2n}(t) = \alpha_{2n}(0) e^{\frac{-4n^2\pi^2\mu}{L^2} t} \\ \beta_{2n}^i(t) = \beta_{2n}^i(0) e^{\frac{-4n^2\pi^2\mu}{L^2} t} \\ \gamma_{2n+1}^i(t) = \gamma_{2n+1}^i(0) e^{\frac{-(2n+1)^2\pi^2\mu}{L^2} t} \end{cases} \quad \text{for all } t \in [0, T]$$

Furthermore, as for all $n \in \mathbb{N}^*, i \in \{1, \dots, I\}$ we have :

$$\rho_0(x) = \sum_{n \in \mathbb{N}^*} \left[\alpha_{2n}(0) V \cos\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^I \beta_{2n}^i(0) e_i \sin\left(\frac{2n\pi x}{L}\right) + \sum_{i=1}^I \gamma_{2n+1}^i(0) e_i \sin\left(\frac{(2n+1)\pi x}{L}\right) \right]$$

with for all $n \in \mathbb{N}^*$:

$$\begin{cases} \alpha_{2n}(0) = \frac{2}{L\|V\|_2^2} \int_0^L \rho_0(x) \cdot V \cos\left(\frac{2n\pi x}{L}\right) dx & \text{for a certain } V_j \neq 0 \\ \beta_{2n}^i(0) = \frac{2}{L} \int_0^L \rho_0(x) \cdot e_i \sin\left(\frac{2n\pi x}{L}\right) dx \\ \gamma_{2n+1}^i(0) = \frac{2}{L} \int_0^L \rho_0(x) \cdot e_i \sin\left(\frac{(2n+1)\pi x}{L}\right) dx \end{cases}$$

The coefficients α, β and γ are imposed by the initial condition. We have therefore found the unique solution to (1).

3 Convergence of the cellular model

3.1 Numerical scheme of the PDE

We are looking for an approximation of the exact solution ρ . We denote $\rho_j^n = \rho(t_n, x_j)$ and $\rho_h^n = (\rho_0^n, \rho_1^n, \dots, \rho_{N-1}^n)^T$

In this section, we impose that P must be irreducible, so that there exists a unique vector V such that :

$$\|V\|_1 = 1, V_i \geq 0 \forall i \text{ and } PV = V$$

Using the expression of the discrete Laplacian and time derivative, we end up with this scheme :

$$\begin{cases} \frac{v_j^{n+1} - v_j^n}{\Delta_t} - \mu \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{\Delta_x^2} = 0, & \forall j \in \{1, \dots, N-1\}, n \in \{0, \dots, M-1\}, \\ \sum_{i=1}^I \left(\frac{v_{1,i}^n - v_{0,i}^n}{\Delta_x} - \frac{v_{0,i}^n - v_{N-1,i}^n}{\Delta_x} \right) = 0, & \forall n \in \{0, \dots, M-1\}, \\ v_0^n = v_N^n, & \forall n \in \{0, \dots, M\}, \\ Pv_0^n = v_0^n, & \forall n \in \{0, \dots, M\}. \end{cases}$$

We can show that it is equivalent to :

$$\forall n \in \{0, \dots, M-1\}$$

$$\begin{cases} v_j^{n+1} = \nu(v_{j-1}^n + v_{j+1}^n) + (1-2\nu)v_j^n & \forall j \in \{1, \dots, N-2\} \\ v_{N-1}^{n+1} = \nu(v_{N-2}^n + v_0^n) + (1-2\nu)v_{N-1}^n \\ v_0^n = \alpha^n V \text{ with } \alpha^n = \frac{1}{2}(\sum_{i=1}^I v_{1,i}^n + v_{N-1,i}^n) \end{cases}$$

3.2 Consistency, Stability and Convergence Analysis

We now show that the cellular model defined in the previous section is consistent and stable in the discrete L^2 norm, and therefore convergent toward the continuous diffusion equation.

Discrete notation. For each time index n , we collect the discrete unknowns in a vector

$$v_h^n = (v_0^n, v_1^n, \dots, v_{N-1}^n)^T \in \mathbb{R}^{IN},$$

where each component $v_j^n \in \mathbb{R}^I$ represents the concentrations of the I layers at node x_j . The discrete L^2 inner product and norm are defined by

$$(u_h, v_h)_h = \Delta_x \sum_{j=0}^{N-1} (u_j^T v_j), \quad \|u_h\|_h = (u_h, u_h)_h^{1/2}.$$

Matrix form of the scheme. The fully discrete explicit scheme can be written compactly as

$$v_h^{n+1} = Qv_h^n, \tag{1}$$

where Q is a block matrix of size $IN \times IN$ that includes:

- the tridiagonal coupling due to horizontal diffusion with coefficients $(\nu, 1-2\nu, \nu)$,
- the boundary connections involving the matrix P .

Consistency. We haven't shown the consistency at this point. Let ρ be a sufficiently exact solution of the continuous diffusion equation. We insert ρ into the discrete scheme to obtain the local truncation (consistency) error:

$$\tau_j^n = \frac{\rho(x_j, t_{n+1}) - \rho(x_j, t_n)}{\Delta_t} - \mu \frac{\rho(x_{j+1}, t_n) - 2\rho(x_j, t_n) + \rho(x_{j-1}, t_n)}{\Delta_x^2}, \quad j = 2, \dots, N-2.$$

For the interior nodes, the Taylor expansion of ρ in t and x around (x_j, t_n) gives

$$\tau_j^n = \frac{\Delta_t}{2} \partial_{tt} \rho(x_j, t_n) - \frac{\mu \Delta_x^2}{12} \partial_{x^4} \rho(x_j, t_n) + \mathcal{O}(\Delta_t^2 + \Delta_x^4), \quad (*)$$

But we haven't shown that this also holds for the boundary nodes. τ_0^n, τ_1^n and τ_{N-1}^n are subject to the same approximation.

Let us assume that $(*)$ holds for every value of j . Hence for a certain $C > 0$ independent of N and M :

$$\|\tau_j^n\|_2 \leq C(\Delta_t + \Delta_x^2) \quad \text{for all } n \in \{0, \dots, M-1\}, j \in \{0, \dots, N-1\}.$$

Therefore,

$$\sup_{0 \leq n \leq M} \|\tau_h^n\|_h \leq C(\Delta_t + \Delta_x^2),$$

and the scheme is **consistent of order** $(1, 2)$.

so the scheme is first order in time and second order in space. This proves the consistency of the method.

Stability in the discrete L^2 norm. It is easy to establish that the matrix Q satisfies

$$\|Q^n\|_2 \leq 1 \quad \text{for all } n \geq 0,$$

provided that the classical CFL condition

$$0 < \nu = \frac{\mu \Delta_t}{\Delta_x^2} \leq \frac{1}{2} \quad (2)$$

is fulfilled. The scheme is stable.

Convergence. Let us define the global error $e_h^n = v_h^n - \rho_h^n$. From the scheme and the consistency relation, we obtain

$$e_h^{n+1} = Q e_h^n + \Delta_t \tau_h^n.$$

Iterating and applying the discrete norm gives

$$\|e_h^{n+1}\|_h \leq \|Q^{n+1}\| \|e_h^0\|_h + \Delta_t \sum_{m=0}^n \|Q^{n-m}\| \|\tau_h^m\|_h.$$

Using $\|Q^k\| \leq 1$ and $e_h^0 = 0$,

$$\|e_h^{n+1}\|_h \leq T \sup_m \|\tau_h^m\|_h \leq C T (\Delta_t + \Delta_x^2).$$

Thus, if it is consistent, the scheme is convergent of order $\mathcal{O}(\Delta_t + \Delta_x^2)$ in discrete L^2 norm.

4 Numerical study of the convergence

Let us examine the convergence of the cellular model for three simple cases: a three-layer basin, with initial functions corresponding to the system's own functions. We will limit ourselves to cases where the coefficients $\alpha_{2k}, \beta_{2k}^i, \gamma_{2k+1}^i$ are zero if and only if $k = 1$. Let us take the simple examples of initial functions corresponding to the system's eigen values. That is functions of the same form as ρ in section 2, with :

- $\alpha_{2k} = \delta_{k,1}, \beta_{2k}^i = 0, \gamma_{2k+1}^1 = \gamma_{2k+1}^2 = \gamma_{2k+1}^3 = 0.$
- $\alpha_{2k} = 0, \beta_{2k}^i = \delta_{k,1}, \gamma_{2k+1}^1 = \gamma_{2k+1}^2 = \gamma_{2k+1}^3 = 0.$
- $\alpha_{2k} = 0, \beta_{2k}^i = \delta_{k,1}, \gamma_3^1 = \gamma_3^2 = 1 \text{ and } \gamma_3^3 = -2\gamma_{2k+1}^1 = \gamma_{2k+1}^2 = \gamma_{2k+1}^3 = 0 \forall k \neq 0.$

5 Mixing Optimization : Optimal Matrix

I have developed a numerical algorithm to model algal growth in a water basin. The model is based on the introduction of a source term into the discretization of the advection–diffusion equation with an upwind discretization of the advection term.

Let us first introduce the numerical scheme. Far from the paddle wheel, the PDE discretization leads to :

$$\forall n \in \{0, \dots, M-1\}$$

$$\frac{v_j^{n+1} - v_j^n}{\Delta t} - D \frac{v_{j+1}^n - 2v_j^n + v_{j-1}^n}{\Delta x^2} + a \frac{v_j^n - v_{j-1}^n}{\Delta x} = 0$$

and the numerical scheme is defined by :

$$\begin{cases} v_j^{n+1} = (\kappa + \nu)v_{j-1}^n + (1 - 2\nu - \kappa)v_j^n + \nu v_{j+1}^n & \forall j \in \{2, \dots, N-2\} \\ v_{N-1}^{n+1} = (\kappa + \nu)v_{N-2}^n + (1 - 2\nu - \kappa)v_{N-1}^n + \nu P v_0^n \\ v_0^{n+1} = (\kappa + \nu)v_{N-1}^n + (1 - 2\nu - \kappa)P v_0^n + \nu v_1^n \\ v_1^{n+1} = (\kappa + \nu)P v_0^n + (1 - 2\nu - \kappa)v_1^n + \nu v_2^n \end{cases}$$

Let us now introduce the source term. Each layer i corresponds to a source term μ_i . Let $\mu = \begin{pmatrix} \mu_1 \\ \vdots \\ \mu_I \end{pmatrix} \in \mathbb{R}^I$.

$$\begin{cases} v_j^{n+1} = (\kappa + \nu)v_{j-1}^n + (1 - 2\nu - \kappa + \mu\Delta t)v_j^n + \nu v_{j+1}^n & \forall j \in \{2, \dots, N-2\} \\ v_{N-1}^{n+1} = (\kappa + \nu)v_{N-2}^n + (1 - 2\nu - \kappa + \mu\Delta t)v_{N-1}^n + \nu P v_0^n \\ v_0^{n+1} = (\kappa + \nu)v_{N-1}^n + (1 - 2\nu - \kappa + \mu\Delta t)P v_0^n + \nu v_1^n \\ v_1^{n+1} = (\kappa + \nu)P v_0^n + (1 - 2\nu - \kappa + \mu\Delta t)v_1^n + \nu v_2^n \end{cases}$$

This additional term modifies the Courant–Friedrichs–Lewy (CFL) condition and, consequently, the value of the time step Δt . To converge, the numerical scheme must verify :

$$\Delta t \leq \frac{\Delta x^2}{2\mu + a\Delta x}$$

5.1 introduction of a biomass source term

In this report, we present first numerical results illustrating the total biomass in the basin after a growth period of $T = 100$ s. According to [?], the algae follow the Han growth model, which is of Haldane if the dynamic of the open state is at its steady state. The light intensity follows a Beer-lambert law [?]:

$$I = I_s e^{-\epsilon z}$$

With $I_s = 1500 \text{ m}^{-2} \mu\text{mol s}^{-1}$

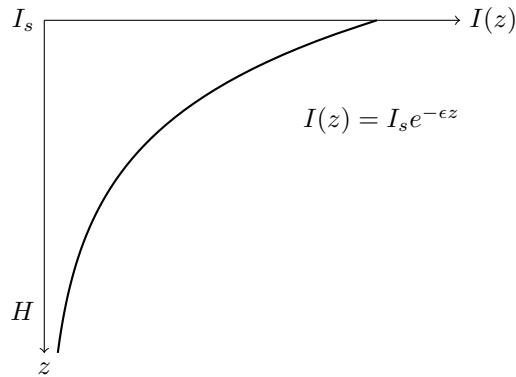


Figure 1: Exponential attenuation of light intensity with depth according to the Beer–Lambert law.

The ϵ light extinction coefficient will be computed once and for all, assuming that the light intensity at depth H is 1% of the surface light intensity I_s . We have :

$$I(H) = \frac{I_s}{100} = I_s \exp(-\epsilon H)$$

Therefore,

$$\epsilon = \frac{\ln(100)}{H} \approx 4.6$$

And for the values of the constants involved in the computing of μ :

$$k_d = 2.99 \times 10^{-5},$$

$$k_r = 4.8 \times 10^{-5} \text{ s}^{-1},$$

$$k_h = 3.64 \times 10^{-5},$$

$$\tau = 6.849 \text{ s},$$

$$\sigma_H = 2.9 \times 10^{-4} \text{ m}^2 \mu\text{mol}^{-1}$$

We have

$$A = \frac{1}{\frac{k_d}{k_r} \tau (\sigma_H I)^2 + \tau \sigma_H I + 1}$$

And

$$\mu = A I k_h \sigma_H = 1.78 \cdot 10^{-4}$$

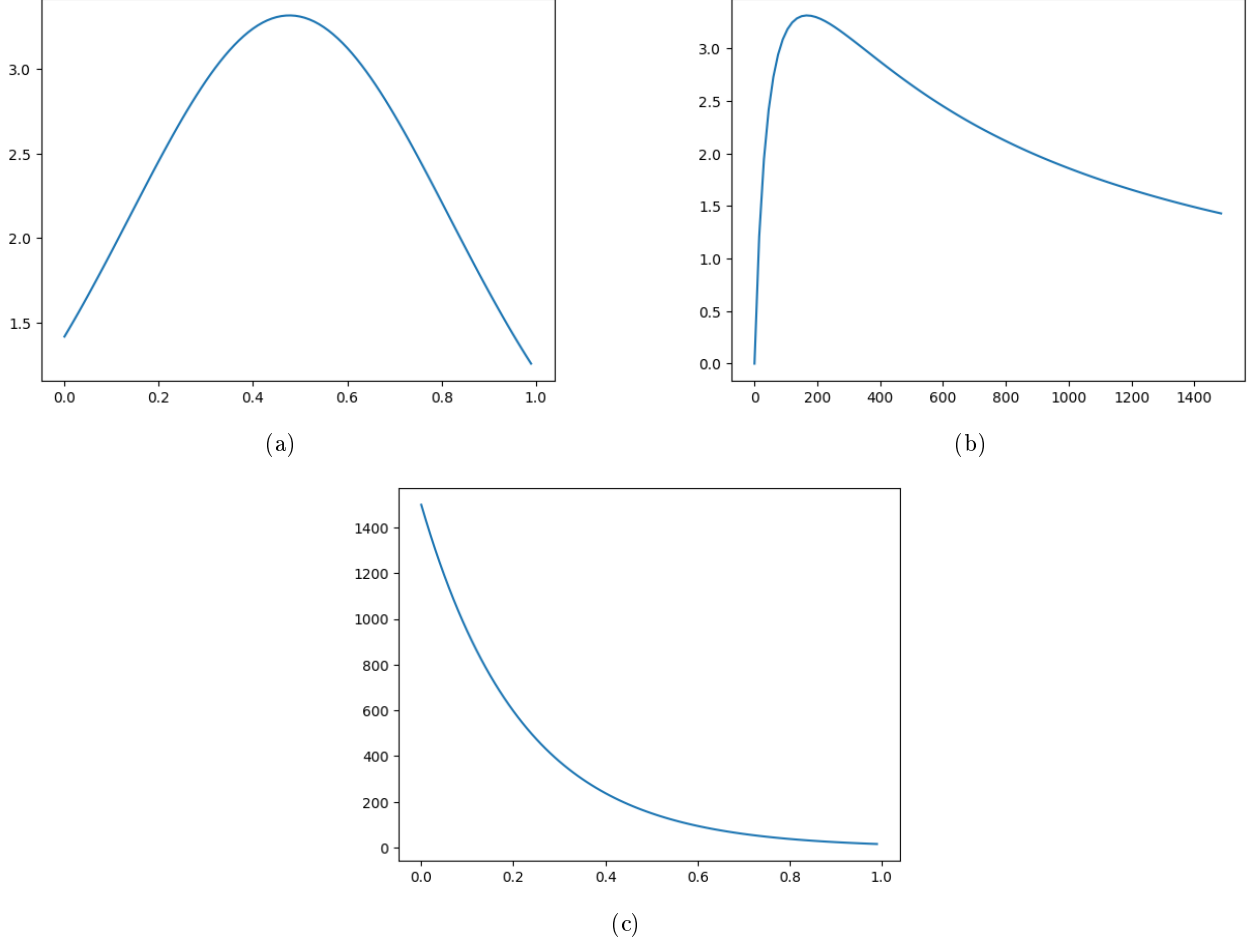


Figure 2: (a) $\mu(d^{-1})$ as a function of z ; (b) $\mu(d^{-1})$ as a function of I ; (c) I as a function of z

The simulations are performed for a single layer, and therefore without any mixing effects. The only varying parameter is the water velocity u . For a discretization of 500 points, we impose a uniform initial concentration and an initial total biomass of 50 Kg. After an hour of growth, we get the following total biomass table for different current speed values.

Current speed (m.s ⁻¹)	Total biomass (kg)
0	52.8041
0.1	52.6224
0.2	52.6222
0.3	52.6221
0.5	52.6221
1	52.6221

Table 1: Total biomass ($T = 3600$ s), as a function of current speed

It seems, that the biomass grows less as the current increases. One must take into account that Δ_t , the time step of the numerical scheme depends on the value of the speed u . This follows from the upwind scheme CFL condition.

6 Discrete Optimization over Permutation Matrices

6.1 Motivation

Before addressing the general optimization problem over stochastic or bistochastic matrices, we first restrict the admissible set to permutation matrices.

This restriction is motivated by both computational and physical considerations. From a computational perspective, the finite cardinality of the set of permutation matrices allows for exhaustive search for small system sizes. From a physical point of view, permutation matrices correspond to deterministic transport of biomass between layers, as opposed to diffusive mixing.

6.2 Problem Definition

The objective of the algorithm is to optimize a mixing matrix P applied to a discretized basin, in order to maximize the total biomass produced by the system. In this section, we restrict P to the set of permutation matrices:

$$\mathcal{P}_N = \{P \in \{0, 1\}^{N \times N} \mid P\mathbf{1} = \mathbf{1}, P^\top \mathbf{1} = \mathbf{1}\}.$$

Given an initial biomass distribution $B(0)$, the system is evolved over a fixed time horizon T using a time-discretized dynamical model (detailed in Section ??). The objective function is defined as the total biomass at final time:

$$J(P) = B_{\text{tot}}(T).$$

The optimization problem reads:

$$\max_{P \in \mathcal{P}_N} J(P).$$

6.3 Exhaustive Search Algorithm

For small values of N , the optimization problem can be solved by exhaustive enumeration of all permutation matrices.

The number of such matrices is $N!$, which rapidly becomes computationally prohibitive as N increases. In practice, exhaustive search is limited to:

$$N \leq 7.$$

The algorithm proceeds as follows:

1. Generate all permutation matrices $P \in \mathcal{P}_N$;
2. For each P , simulate the system dynamics up to time T ;
3. Compute the objective value $J(P)$;
4. Select the matrix maximizing J .

6.4 Tested Configurations

The total initial biomass is fixed:

$$B_{\text{tot}}(0) = 50 \text{ Kg.}$$

The growth time is fixed to:

$$T = 300 \text{ s,}$$

the flow velocity is set to:

$$u = 0.1 \text{ m s}^{-1},$$

and the basin length is set to:

$$L = 5 \text{ m.}$$

300 seconds of growth therefore correspond to approximately 6 laps.

To assess the influence of initial conditions, several configurations are considered :

- biomass entirely localized in a single layer, while all other layers are initialized with zero biomass. This leads to N distinct initial configurations.
- Uniform distribution of the biomass.

6.5 Results for Small Systems

6.5.1 Case $N = 3$

The growth rates associated with each layer are given in Table ??.

Layer	$\mu \text{ (s}^{-1}\text{)}$	Light intensity
Layer 1	1.4198×10^{-5}	1500
Layer 2	3.0538×10^{-5}	323.72
Layer 3	2.8777×10^{-5}	69.86

Table 2: Growth rates and light intensities for $N = 3$

For each initial condition, the optimal permutation matrix is determined. The results are summarized below.

- Configuration 1 (biomass initially in layer #1):
Optimal matrix:

$$\mathbf{P}_1 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Final biomass:

$$B_{\text{tot}} = 50.369 \text{ Kg}$$

This matrix induces a permutation of layers which induces the allocation of layer 1 in position 2.

- Configuration 2 (biomass initially in layer #2):
Optimal matrix:

$$\mathbf{P}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Final biomass:

$$B_{\text{tot}} = 50.461 \text{ Kg}$$

Here, no mixing appears to be the optimal strategy.

- Configuration 3 (biomass initially in layer #3):
Optimal matrix:

$$\mathbf{P}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Final biomass:

$$B_{\text{tot}} = 50.447 \text{ Kg}$$

This configuration corresponds to the permutation of layers 2 and 3.

- Configuration 4 : uniform distribution
Optimal matrix:

$$\mathbf{P}_4 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Final biomass:

$$B_{\text{tot}} = 50.370 \text{ Kg}$$

As in case 2, no mixing appears to be the optimal strategy.

6.6 Discussion

The results reveal a strong dependence of the optimal strategy on the initial configuration.

When biomass is initially located in a layer with high growth rate, the identity permutation (i.e., no mixing) is optimal. We observe the same phenomenon for a uniform initial distribution. In contrast, when biomass is initially located in less favorable layers, the optimal permutations transport biomass toward layers with higher growth rates. For example, in configuration 1, if line i has the highest growth coefficient, the optimal permutation matrix always verifies $P_{i,1} = 1$. This means that the first layer, initially containing all the biomass is allocated to the region of highest growth. This behavior is observed regardless of the initial condition, although the optimal permutation depends on the initial localization of the biomass. It is consistent with the physical interpretation of the system: the mixing strategy aims at maximizing the effective light exposure experienced by the biomass over time.

These observations will serve as a basis for the analysis of more general mixing strategies in the following sections.

However, as time passes, the mixing matrix makes the basin more and more homogenous. As supposed earlier, we notice that the mixing tend to equalize the algae concentration in each layer. Here is a graphic representing the total algae mass in each layer as a function of time, for configuration 3 and using the mixing matrix $\mathbf{P}_3$.

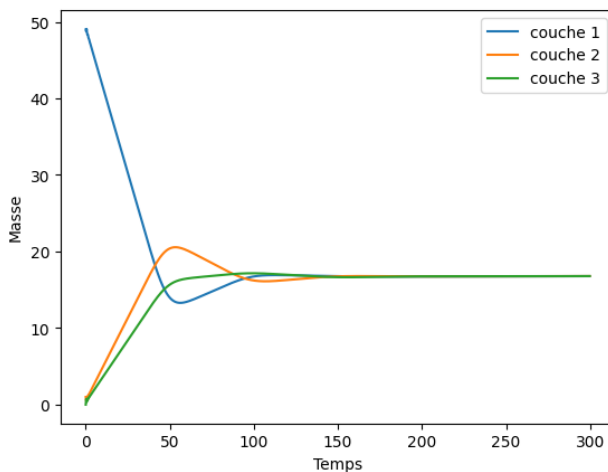


Figure 3: biomass in each layer through time

The vector of concentrations in the column corresponding to the paddle converges to a multiple of an eigen vector of the matrix. This phenomenon has been observed with different initial conditions, and with N varying from 3 to 7.

6.7 Interpretation in Terms of Biomass Trajectories

A permutation matrix defines a deterministic mapping between layers. Iterating such a matrix over time induces periodic trajectories for each biomass element.

Therefore, instead of interpreting the mixing process as a spatial redistribution, it is natural to adopt a Lagrangian viewpoint and consider the temporal trajectories experienced by individual biomass particles.

Under this interpretation, a permutation matrix generates cyclic trajectories:

$$i \mapsto P(i) \mapsto P^2(i) \mapsto \dots$$

The growth of each biomass element is then governed by the sequence of light intensities encountered along its trajectory.

7 Connection with the Flashing Light Effect

We have observed an interesting phenomenon by modifying the basin length.

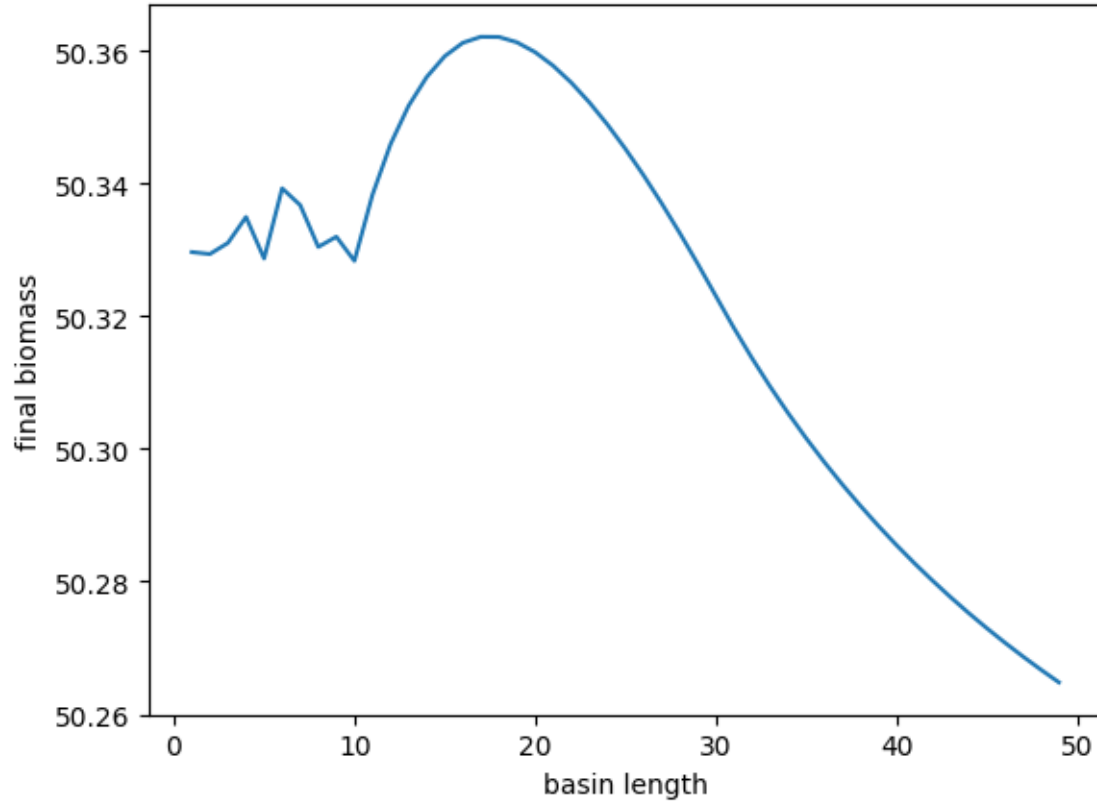


Figure 4: optimal biomass as a function of L

For the same parameters, excepted the length L , the algae tend to grow more when the length L decreases. However, for smaller values of L ($L \leq 20$), the total production of biomass becomes an increasing function of L . Let us notice that as L gets smaller, the number of mixes for each algae tends to increase. This phenomenon is called the flashing effect, and has been observed and analyzed in....

As the number of mixes increases, each algae particule gets to see a higher intensity (at the surface of the pond) more frequently. Therefore, as this frequency varies, the percentage of PSU in each state (inhibited, open and closed) also varies. This brings us to study how exactly does this frequency influences the biomass production. In fact, as shown in the photosynthetic response is nonlinear with respect to light intensity. In particular:

- high light intensities may lead to saturation or photoinhibition,
- low light intensities result in suboptimal growth,
- intermittent exposure can enhance overall efficiency.

In our work, the observed structure of optimal permutations can be directly related to the flashing light effect. This effect suggests that the optimization process naturally balances two competing effects:

- maximizing exposure to favorable conditions (high-growth layers),
- avoiding constant exposure that would reduce efficiency.

In Table ??, the optimal length seems to be $L = 17$ m. For computational reasons, we will keep the length at $L = 5$ m.

Permutation-based mixing will be later compared to a more general case : bistochastic-based mixing. In fact, permutation-based mixing induces more alternations between different light environments than the latter. This indicates that permutation matrices could bring a higher performance than bistochastic matrices. This will be looked through later.

8 Relaxation to Stochastic Mixing Matrices

8.1 Motivation

Now that we have found a way to determine the best permutation matrix for the water pond, let us now focus on a more general case : the column stochastic matrices. To optimize on this infinite space, we will have to use a differentiable program. We have chosen the torch python library to do so. This will allow us to compute torch differentiation with these variables.

8.2 Admissible Set and Parametrization

We consider the set of column-stochastic matrices:

$$\mathcal{S}_N = \left\{ P \in \mathbb{R}^{N \times N} \mid P_{ij} \geq 0, \sum_{i=1}^N P_{ij} = 1 \right\}.$$

In order to handle these constraints within a differentiable optimization framework, we introduce an unconstrained matrix $Q \in \mathbb{R}^{N \times N}$ and define:

$$P_{ij} = \frac{\exp(Q_{ij})}{\sum_{k=1}^N \exp(Q_{kj})}.$$

This parametrization ensures that P remains column-stochastic for any Q , while allowing unconstrained optimization in $\mathbb{R}^{N \times N}$.

8.3 Optimization Procedure

The objective function remains unchanged:

$$J(P) = B_{\text{tot}}(T).$$

The optimization is performed using a gradient-based method applied to the parameter matrix Q .

Gradients are computed through automatic differentiation, using a differentiable implementation of the dynamical model.

The update rule is given by:

$$Q \leftarrow Q + \alpha \nabla_Q J,$$

where α denotes the learning rate.

In practice, we employ the Adam optimizer, which provides adaptive step sizes and improves convergence properties.

8.4 Numerical Setup

The relaxation allows us to consider larger systems, typically:

$$N = 10.$$

The initial biomass is concentrated in a single layer:

$$B(0) = 50 \text{ Kg.}$$

The time horizon and flow parameters are identical to those used in the discrete optimization:

$$T = 300 \text{ s}, \quad u = 0.1 \text{ m.s}^{-1} 0.1 \text{ m.s}^{-1} 0.1 \text{ m.s}^{-1} 0.1 \text{ m.s}^{-1}.$$

The optimization is initialized from:

- either a random matrix Q ,
- or a structured initialization (to be specified).

8.5 Optimal Stochastic Matrix

The algorithm complexity is a linear function of the number of layers (cf annexes). Therefore, it allows us to divide the water pond into a larger number of layers.

For $N = 10$, the optimization yields the following matrix:

$$P^* = \begin{pmatrix} 0.000 & 0.003 & 0.010 & 0.000 & 0.000 & 0.000 & 0.001 & 0.001 & 0.001 & 0.004 \\ 0.000 & 0.011 & 0.001 & 0.020 & 0.000 & 0.000 & 0.000 & 0.007 & 0.004 & 0.006 \\ 0.000 & 0.009 & 0.008 & 0.003 & 0.000 & 0.000 & 0.000 & 0.005 & 0.004 & 0.004 \\ 0.001 & 0.394 & 0.373 & 0.080 & 0.000 & 0.000 & 0.002 & 0.112 & 0.151 & 0.082 \\ 0.003 & 0.285 & 0.251 & 0.072 & 0.012 & 0.000 & 0.504 & 0.073 & 0.193 & 0.051 \\ 0.992 & 0.095 & 0.076 & 0.443 & 0.985 & 0.999 & 0.345 & 0.511 & 0.331 & 0.599 \\ 0.003 & 0.087 & 0.140 & 0.371 & 0.001 & 0.000 & 0.142 & 0.283 & 0.268 & 0.203 \\ 0.000 & 0.103 & 0.131 & 0.005 & 0.001 & 0.000 & 0.005 & 0.001 & 0.041 & 0.045 \\ 0.000 & 0.007 & 0.005 & 0.006 & 0.000 & 0.000 & 0.000 & 0.001 & 0.001 & 0.002 \\ 0.000 & 0.007 & 0.006 & 0.000 & 0.000 & 0.000 & 0.000 & 0.005 & 0.005 & 0.002 \end{pmatrix}$$

As seen before, the line 6 has the largest coefficients because layer 6 has the highest growth coefficient. The optimal matrix allocates most of the biomass in this layer. But stochastic matrices aren't physically relevant. Indeed, in terms of water volume, a real-life paddle wheel should allocate to each layer exactly the volume of one layer. Such a wheel could only be represented by a bistochastic matrix.

9 Optimization under Bistochastic Constraints

9.1 Motivation

While stochastic matrices provide a convenient relaxation of the original problem, they lack a clear physical interpretation in the context of mixing processes.

Indeed, a realistic mixing mechanism (such as a paddle wheel) must conserve the volume of each layer. This implies that the mixing matrix must preserve both column sums and row sums.

This motivates restricting the admissible set to bistochastic matrices:

$$\mathcal{B}_N = \left\{ P \in \mathbb{R}^{N \times N} \mid P_{ij} \geq 0, \sum_{i=1}^N P_{ij} = 1, \sum_{j=1}^N P_{ij} = 1 \right\}.$$

From a physical perspective, bistochastic matrices represent volume-preserving redistribution of biomass across layers.

9.2 Optimization Strategy

The optimization problem remains:

$$\max_{P \in \mathcal{B}_N} J(P).$$

In order to handle the additional row constraints, we adopt a penalized formulation.

Starting from the softmax parametrization used for stochastic matrices, we introduce a penalty term enforcing row normalization:

$$\tilde{J}(P) = J(P) - \lambda \sum_{i=1}^N \left(\sum_{j=1}^N P_{ij} - 1 \right)^2,$$

where $\lambda > 0$ is a penalty parameter.

The optimization is then performed over the unconstrained matrix Q , as in the previous section.

As the objective function is non-convex, the obtained solutions may depend on initialization and may correspond to local optima.

9.3 Numerical Results

After launching the program for a 10 layers pond, we end up with the following matrix.

$$\begin{bmatrix} 0.1727 & 0.0401 & 0.1280 & 0.1118 & 0.0686 & 0.0839 & 0.0447 & 0.2331 & 0.0285 & 0.0885 \\ 0.0422 & 0.2675 & 0.0660 & 0.1765 & 0.0297 & 0.0281 & 0.1194 & 0.1010 & 0.0907 & 0.0790 \\ 0.2268 & 0.0297 & 0.1286 & 0.0654 & 0.0687 & 0.0290 & 0.2182 & 0.0791 & 0.1469 & 0.0075 \\ 0.1335 & 0.1178 & 0.0531 & 0.0894 & 0.1766 & 0.0053 & 0.0866 & 0.0164 & 0.0864 & 0.2348 \\ 0.2559 & 0.0255 & 0.3115 & 0.0160 & 0.0122 & 0.0688 & 0.0067 & 0.1059 & 0.1850 & 0.0123 \\ 0.0238 & 0.3348 & 0.0341 & 0.0567 & 0.2197 & 0.0715 & 0.0415 & 0.0387 & 0.1417 & 0.0375 \\ 0.0467 & 0.0217 & 0.0830 & 0.3417 & 0.0783 & 0.0183 & 0.2618 & 0.0856 & 0.0249 & 0.0380 \\ 0.0410 & 0.0510 & 0.0688 & 0.0451 & 0.2185 & 0.1392 & 0.0301 & 0.2752 & 0.1097 & 0.0213 \\ 0.0356 & 0.0181 & 0.0586 & 0.0700 & 0.1100 & 0.5202 & 0.0761 & 0.0173 & 0.0743 & 0.0198 \\ 0.0218 & 0.0936 & 0.0683 & 0.0274 & 0.0177 & 0.0357 & 0.1148 & 0.0476 & 0.1119 & 0.4613 \end{bmatrix}$$

This mixing matrix corresponds to a final biomass (after $T = 300s$ of growth) of $B_{\text{tot}} = 50.369 \text{ Kg}$.

9.4 Comparison with Permutation Matrices

The previous optimal permutation matrix algorithm allows us to compare the best permutation matrix to the bistochastic case. A key question is whether relaxing the admissible set from permutation matrices to bistochastic matrices leads to improved performance.

9.5 Local Optimality of Permutation Matrices

To further investigate the relationship between permutation and bistochastic matrices, we perform the following experiment:

- initialize the gradient-based optimization at an optimal permutation matrix,
- run the optimization under bistochastic constraints.

In all tested cases, the algorithm converges to a plateau without departing significantly from the initial permutation matrix.

In other words, small perturbations of a permutation matrix within $\mathcal{B}_N$ do not lead to an increase in the objective function. This suggests that, for a given permutation matrix P^* , there exists a neighborhood $\mathcal{U} \subset \mathcal{B}_N$ such that:

$$J(P) \leq J(P^*), \quad \forall P \in \mathcal{U}.$$

In other words, permutation matrices behave as locally optimal solutions within the admissible set.

9.6 Interpretation

Although the admissible set has been extended to include all physically relevant mixing strategies, the optimal solutions remain located near extremal points of the feasible set.

In fact, what the simulations seem to indicate, is that the optimal permutation matrices are at least a local maximum of the objective function on the set of bistochastic matrices. The bistochastic optimization does not produce solutions that significantly outperform the best permutation matrices. The results indicate that:

$$J_{\text{bistochastic}} \leq J_{\text{permutation}} \quad (\text{observed numerically}).$$

9.6.1 Extremal Nature of Optimal Strategies

An important observation is that permutation matrices correspond to extremal points of the set of bistochastic matrices.

Indeed, any bistochastic matrix can be written as a convex combination of permutation matrices:

$$P = \sum_k \lambda_k P_k, \quad \lambda_k \geq 0, \quad \sum_k \lambda_k = 1.$$

From this perspective, stochastic or bistochastic mixing can be interpreted as a probabilistic combination of deterministic trajectories.

The numerical results suggest that such averaging effects are detrimental to the objective function. Indeed, spreading biomass over multiple trajectories tends to reduce the temporal contrast in light exposure, thereby weakening the flashing light effect.

In contrast, permutation matrices preserve fully coherent trajectories, which appear to be more favorable for growth.

9.7 Heuristic Explanation of Optimality

This phenomenon can be interpreted in light of the structure of the objective function.

The function $J(P)$ depends on the temporal evolution of biomass under repeated application of the mixing matrix. As discussed earlier, growth is enhanced by temporal variability in light exposure. Optimal mixing strategies must therefore maximize the temporal variability of light exposure while ensuring frequent visits to high-growth regions.

In a Lagrangian point of view, coherent and structured trajectories for each algae maximize this variability. These types of trajectories are induced by permutation matrices.

In contrast, bistochastic matrices introduce averaging effects by splitting biomass across multiple pathways. This reduces the temporal variability of light exposure, thereby weakening the flashing light effect.

As a consequence, any local perturbation of a permutation matrix in a direction that increases mixing tends to decrease the objective function.

9.8 Conclusion

The bistochastic optimization does not lead to significant improvements over permutation-based strategies, despite the larger admissible set.

Combined with the structural insights obtained previously, this suggests that optimal mixing strategies are inherently extremal.

This observation will be further discussed in the final section, where we address the implications and limitations of these findings.

10 Towards Global Optimality

While the local optimality of permutation matrices is strongly supported by numerical evidence, establishing global optimality remains a challenging problem.

Indeed, the objective function J is highly nonlinear and non-convex, due to the coupling between transport dynamics and biological growth.

Therefore, standard convex optimization arguments cannot be directly applied.

Nevertheless, several observations suggest that permutation matrices may also be globally optimal:

- exhaustive search for small values of N consistently identifies permutations as global maximizers;
- gradient-based optimization over larger admissible sets does not produce better-performing solutions;

10.1 Conjecture

Based on these observations, we formulate the following conjecture:

For the class of models considered in this work, the global maximizers of the objective function over the set of bistochastic matrices are permutation matrices.

This conjecture is supported by both numerical evidence and physical arguments, but a rigorous proof remains an open question.

10.2 Limitations and Perspectives

Several limitations must be acknowledged.

First, the numerical exploration of the admissible set remains partial for large values of N , and the possibility of better solutions cannot be completely ruled out.

Second, the optimization process may converge to local maxima depending on initialization, although consistent results were obtained across multiple runs.

Future work could focus on:

- developing analytical tools to characterize the structure of the objective function,
- establishing sufficient conditions for global optimality of permutation matrices,
- exploring connections with optimal transport and control theory.

11 annexes

11.1 initial test

To check if the new solver is functional. In order to do so, let us consider a three layers basin, and an objective function which mixes these layers with respect to the matrix $P = \text{softmax}(Q)$ and returns the total biomass, after a pre-determined growing period. The growth coefficients will be arbitrarily set to 1 for layer 1, 2 for layer 2 and 3 for layer 3. Therefore, with an initial basin in which the total biomass is concentrated in layer 1, an optimal matrix should reallocate it in layer 3. Let us consider the results.

The optimal mixing matrix is:

$$\mathbf{P} = \begin{pmatrix} 0.003 & 0.50 & 0.26 \\ 0.007 & 0.30 & 0.10 \\ 0.99 & 0.20 & 0.64 \end{pmatrix}$$

The first layer is indeed moved to layer 3 by 99 percent.

11.2 3 layers comparison test

With an initial pond of 3 layers, in which the biomass is only in the first layer, the solver returns the following matrix :

$$\begin{pmatrix} 4.126 \times 10^{-5} & 3.704 \times 10^{-4} & 2.707 \times 10^{-4} \\ 9.961 \times 10^{-1} & 9.976 \times 10^{-1} & 9.848 \times 10^{-1} \\ 3.901 \times 10^{-3} & 2.034 \times 10^{-3} & 1.495 \times 10^{-2} \end{pmatrix}$$

this matrix corresponds to a situation in which the final biomass is 50.301Kg. This is approximately the same result than the permutation matrix case. The optimal permutation matrix was indeed :

$$\mathbf{M}_1 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

We can now begin the ADAM algorithm starting from this matrix and see if we obtain a better result. The final biomass result stays the same, while the optimisation returns a new matrix :

$$\begin{pmatrix} 7.768 \times 10^{-4} & 3.514 \times 10^{-4} & 3.007 \times 10^{-3} \\ 9.933 \times 10^{-1} & 9.974 \times 10^{-1} & 9.429 \times 10^{-1} \\ 5.955 \times 10^{-3} & 2.301 \times 10^{-3} & 5.411 \times 10^{-2} \end{pmatrix}$$

11.3 Newton method

Let us try to use another optimizer, based on the Newton method with conjugate gradient. This method is a second-order optimization technique used to find a local minimizer of a twice-differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$. At iteration k , the Newton direction p_k is defined as the solution of the linear system

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k),$$

where $\nabla f(x_k)$ is the gradient and $\nabla^2 f(x_k)$ is the Hessian matrix.

For large-scale problems as ours, explicitly forming and inverting the Hessian matrix is computationally expensive. Instead, the Newton system can be solved approximately using the Conjugate Gradient (CG) method. The CG algorithm is particularly efficient when the Hessian is symmetric and positive definite, as it only requires matrix-vector products.

In the Newton-CG method, at each iteration we apply the Conjugate Gradient algorithm to compute an approximate solution p_k of the Newton system. The iterate is then updated as

$$x_{k+1} = x_k + \alpha_k p_k,$$

where α_k is determined by a line search procedure.

11.4 Computational complexity.

We analyze the computational complexity of the optimization procedure by first estimating the cost of a single evaluation of the objective function. The time step of the numerical scheme is given by

$$\Delta t = \text{CFL} \frac{\Delta x^2}{2\mu + a \Delta x}, \quad \text{with} \quad \Delta x = \frac{L}{n},$$

where n denotes the number of spatial discretization points, μ the diffusion coefficient, and a a characteristic advection velocity. Substituting Δx yields

$$\Delta t = \text{CFL} \frac{L^2}{n^2(2\mu + aL/n)} = \frac{\text{CFL} L^2}{2\mu n^2 + aLn}.$$

In our setting, typical parameter values are $\mu \approx 10^{-5}$, $n \approx 50$, $0.01 < a < 0.1$, and $L \approx 5$. Under these assumptions, the term aLn dominates $2\mu n^2$, leading to the approximation

$$\Delta t \sim \frac{\text{CFL} L}{a n}.$$

The objective function corresponds to the final state of a time-marching simulation over a time horizon T . The number of time steps is therefore of order $T/\Delta t$, and each time step involves $O(Nn)$ operations, where N denotes the number of layers. Consequently, the computational complexity of a single forward evaluation of the objective function is

$$\mathcal{O}\left(Nn \frac{T}{\Delta t}\right) = \mathcal{O}\left(\frac{a N n^2 T}{\text{CFL} L}\right).$$

Since the objective function is implemented in a differentiable manner using automatic differentiation, the backward pass requires propagating gradients through the entire time-marching scheme. The computational cost of the backward pass is of the same order as the forward pass, up to a constant factor, yielding

$$\mathcal{O}\left(\frac{a N n^2 T}{L}\right).$$

Finally, the full optimization procedure consists of K iterations of the Adam algorithm. Each iteration requires one forward evaluation of the objective function and one backward pass, while the update of the parameter matrix $P \in \mathbb{R}^{N \times N}$ is negligible in comparison. The overall computational complexity of the `solve` algorithm is therefore

$$\mathcal{O}\left(K \frac{a N n^2 T}{L}\right).$$

11.4.1 parameters of the Adam optimizer

The Adam optimizer (Adaptive Moment Estimation) combines momentum-based gradient descent with per-parameter adaptive learning rates. At each iteration t , it maintains an exponential moving average of the gradient m_t (first moment) and of the squared gradient v_t (second moment), defined as

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

where g_t denotes the gradient of the loss with respect to the parameters. These moment estimates are corrected for initialization bias as

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The parameter update is then given by

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},$$

where α is the global learning rate, while β_1 and β_2 control the exponential decay rates of the first and second moments, respectively, governing the trade-off between stability and responsiveness of the optimization process.

In order to choose an appropriate learning rate for the Adam optimization algorithm, we performed a learning rate finder experiment. The principle consists in launching the optimization with different α values, and check which one converges the quickest.

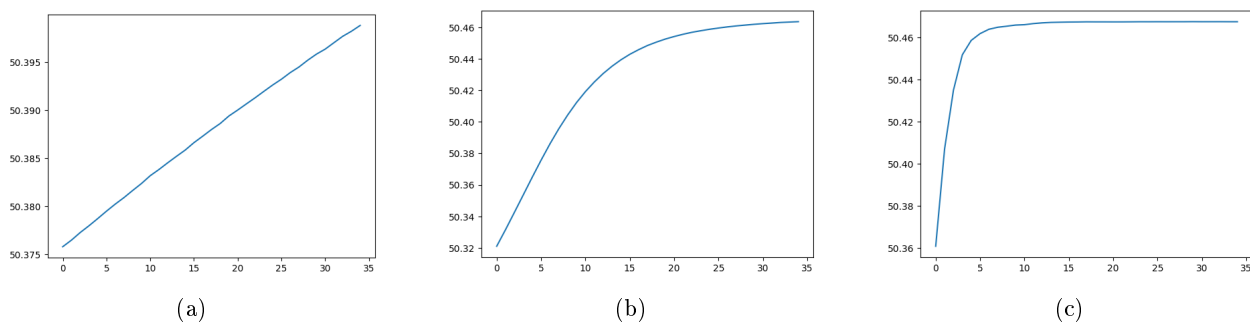


Figure 5: (a) $\alpha = 0.01$; (b) $\alpha = 0.1$; (c) $\alpha = 0.9$

It is clear that the best learning rate is $\alpha = 0.9$. It reaches a plateau after about 12 iterations. Based on this observation, we can arbitrarily bond the number of iterations by 12.

References

- [1] Liu-Di Lu, *Lagrangian Approaches for Modelling and Optimization of Hydrodynamic-Photosynthesis Coupling*, Sorbonne Université, LJLL, 2021
- [2] Oxford reference *Beer-Lambert Law*